Vision-Cued Minimum-Jerk Collision Avoidance for a Wind-Perturbed Quadrotor:

A Mathematical and Algorithmic Reconstruction of the DroneSim Pipeline — Rigid-Body Dynamics, Geometric Cascaded Control, Disturbance Observation, Ornstein–Uhlenbeck Gust Synthesis and Monocular Time-to-Contact Estimation

Technical report derived from source-code analysis of the *Drone Navigator Telemetry System* repository

October 16, 2026

Abstract

This report documents, equation by equation, what the simulator in the repository *actually* computes. It is based on the source (`dronesim/`, `frontend/src`, `tests/`) and not on the README, which describes only an early prototype (a YOLO heads-up display with a heuristic distance $d = 3000/(h_{\text{box}} + 1)$). The current system is a full pipeline: a monocular video is passed through a detector/tracker; the apparent growth of the tracked bounding box gives a time-to-contact (TTC) and, with an assumed physical size and a pinhole camera model, a metric range, closing speed and lateral offset; these define a static-obstacle scenario; a sampling-based planner chooses a minimum-jerk detour (or a braking stop) subject to acceleration, thrust and tilt limits that include aerodynamic drag in a steady 3-D wind; and a cascaded geometric flight controller with a Luenberger disturbance observer tracks the plan through a 17-state, RK4-integrated quadrotor model with four first-order rotors, in a world that may contain seeded Ornstein–Uhlenbeck turbulence. The result is recorded in a deterministic JSON schema and replayed in a web front end. All parameters are *assumed*, not measured; the report flags these assumptions and lists observed limitations in the final section.

Contents

1 Scope, Provenance and the State of the README

The repository contains two generations of code.

Generation 1 (legacy; what the README describes).
`object_detection_for_rescue_operations.py` runs YOLOv8 on each frame, draws boxes, declares a target "LOCKED" when the box centre lies in the middle 20 % of the image ($0.4W < c_x < 0.6W$), and displays a *heuristic* distance

$$\widehat{d} \;=\; \mathrm{round}\Big(\frac{3000}{(y_2 - y_1) + 1},\, 1\Big) \text{ “m”}, \tag{1}$$

which is an uncalibrated inverse-proportional mapping (the constant 3000 has units px·m and is not derived from any camera model). There is no dynamics, planning or control.

This report concerns Generation 2. Section **??** lists the points where the code departs from what a casual reading of the README (or of some docstrings) would suggest.

2 System Architecture

2.1 End-to-end data flow

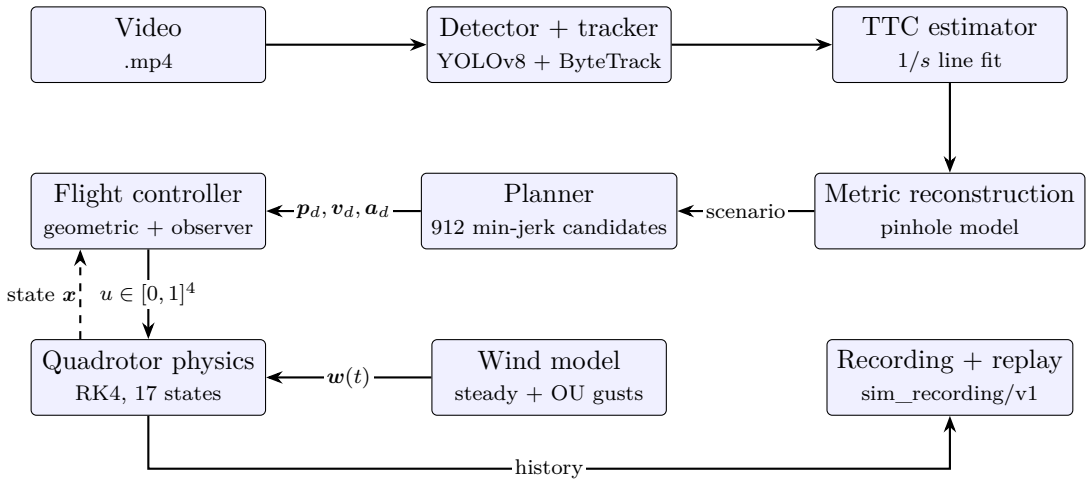


Figure 1: Pipeline. Solid arrows: data flow. Dashed: the controller reads the true simulated state (no sensor noise).

2.2 Module map

Module	Role
`config.py`	Dataclasses for vision, obstacle size and camera FOV (all flagged *assumed*).
`vision/detector.py`	YOLO + tracker wrapper; yields detections with t = frame/fps.
`vision/obstacle_estimator.py`	Bounding box → size signal $s = \sqrt{wh}$, TTC and growth by least squares.
`vision/reconstruction.py`	Pixels → metres (range, lateral, vertical, closing speed); builds the scenario.
`vision/export.py`	Target-track selection, writes `obstacle_track.json`.
`sim/params.py`	Drone parameters and derived constants (k_T, k_M, hover command, motor layout).
`sim/quaternion.py`	$[w, x, y, z]$ quaternion algebra, Euler conversions.
`sim/motors.py`	Command→ ω → thrust/torque/power maps; mixer matrix.
`sim/quadrotor.py`	17-state dynamics, RK4, zero-order-hold command.
`sim/wind.py`	Steady wind vector + seeded Ornstein–Uhlenbeck gusts.
`sim/collision.py`	Box signed distance, clearance, collision prediction.
`sim/planner.py`	Min-jerk detour/brake candidate generation, wind-aware feasibility, cost.
`sim/controller.py`	Cascaded controller, disturbance observer, desaturating mixer.
`sim/loop.py`	Multi-rate closed-loop driver (physics 500 Hz, control 250 Hz).
`sim/runner.py`	Plan → fly → verify → re-plan; summaries.
`sim/recording.py`	JSON schema, random-access `Replay`.
`api/app.py`	FastAPI service: runs, video, re-simulation without re-running YOLO, async processing jobs.
`frontend/src/*`	React/Three.js replay (HUD, motor graphs, wind widgets); TypeScript port of `Replay`.

2.3 Execution sequence

The CLI chain (also invoked by the API as three subprocesses) is

(i) `python -m dronesim.vision.export VIDEO` → `obstacle_track.json` (schema `obstacle_track/v1`);

(ii) `python -m dronesim.vision.reconstruction obstacle_track.json` → `obstacle_metric.json` (`obstacle_metric/v1`);

(iii) `python -m dronesim.run_scenario obstacle_metric.json [wind, dodge, ...]` → `sim_recording.json` (`sim_recording/v1`).

The API endpoint `POST /api/runs/{run}/simulate` redoes only step (iii) with new settings, so YOLO is never re-run.

3 Conventions and Notation

- **World frame** $\mathcal{W}$ (ENU-like): $+x$ is the drone's initial forward / approach axis, $+y$ is left, $+z$ is up.
- **Body frame** $\mathcal{B}$ (FLU): $+x$ forward, $+y$ left, $+z$ up.
- **Camera frame**: $+Z$ forward (range), $+X$ right, $+Y$ up. The conversion to the world frame is $(Z, X, Y) \mapsto (x, -y, z)$, i.e. camera-right becomes world $-y$.
- **Quaternion** $\boldsymbol{q} = [q_w, q_x, q_y, q_z]^\top$, unit, rotating body vectors to world vectors: $\boldsymbol{v}_\mathcal{W} = \boldsymbol{R}(\boldsymbol{q})\boldsymbol{v}_\mathcal{B}$.
- **Euler angles**: ZYX (yaw–pitch–roll) in radians.

Symbol	Meaning	Symbol	Meaning
$\boldsymbol{p}, \boldsymbol{v}$	position, velocity in $\mathcal{W}$	$\boldsymbol{\omega}$	body angular velocity
$\boldsymbol{R}, \boldsymbol{q}$	body→world rotation / quaternion	Ω_i	rotor i speed (rad/s)
m	mass	$\boldsymbol{J}$	inertia matrix
T_i	thrust of motor i	$\boldsymbol{w}(t)$	wind velocity in $\mathcal{W}$
c_d	quadratic drag coefficient (kg/m)	$\boldsymbol{d}$	disturbance (drag) acceleration
k_T, k_M	thrust / torque coefficients	$u_i \in [0,1]$	normalised motor command
$\boldsymbol{F}$	desired force vector	g	$9.80665\,\mathrm{m/s^2}$

Table 1: Principal symbols.

3.1 Quaternion algebra (`quaternion.py`)

Hamilton product for $\boldsymbol{a} = [a_w, \boldsymbol{a}_v]$, $\boldsymbol{b} = [b_w, \boldsymbol{b}_v]$:

$$\boldsymbol{a} \otimes \boldsymbol{b} = \big[a_w b_w - \boldsymbol{a}_v \cdot \boldsymbol{b}_v,\ a_w \boldsymbol{b}_v + b_w \boldsymbol{a}_v + \boldsymbol{a}_v \times \boldsymbol{b}_v\big].$$

Rotation matrix:

$$\boldsymbol{R}(\boldsymbol{q}) = \begin{bmatrix} 1-2(q_y^2+q_z^2) & 2(q_xq_y - q_zq_w) & 2(q_xq_z + q_yq_w) \\ 2(q_xq_y + q_zq_w) & 1-2(q_x^2+q_z^2) & 2(q_yq_z - q_xq_w) \\ 2(q_xq_z - q_yq_w) & 2(q_yq_z + q_xq_w) & 1-2(q_x^2+q_y^2) \end{bmatrix}. \tag{2}$$

Euler → quaternion (half-angles $c_\bullet = \cos(\bullet/2)$, $s_\bullet = \sin(\bullet/2)$):

$$\begin{aligned} q_w &= c_r c_p c_y + s_r s_p s_y, & q_x &= s_r c_p c_y - c_r s_p s_y, \\ q_y &= c_r s_p c_y + s_r c_p s_y, & q_z &= c_r c_p s_y - s_r s_p c_y. \end{aligned}$$

Quaternion → Euler:

$$\phi = \operatorname{atan2}\big(2(q_w q_x + q_y q_z),\ 1 - 2(q_x^2 + q_y^2)\big), \tag{3}$$

$$\theta = \arcsin\big(\operatorname{clip}(2(q_w q_y - q_z q_x), -1, 1)\big), \tag{4}$$

$$\psi = \operatorname{atan2}\big(2(q_w q_z + q_x q_y),\ 1 - 2(q_y^2 + q_z^2)\big). \tag{5}$$

Rotation matrix → quaternion uses Shepperd's method: if $\operatorname{tr}\boldsymbol{R} > 0$, $s = 2\sqrt{\operatorname{tr}\boldsymbol{R} + 1}$ and $\boldsymbol{q} = \big[\frac{s}{4}, \frac{R_{32}-R_{23}}{s}, \frac{R_{13}-R_{31}}{s}, \frac{R_{21}-R_{12}}{s}\big]$; otherwise the largest diagonal element i is used (indices j, k cyclic), $s = 2\sqrt{R_{ii} - R_{jj} - R_{kk} + 1}$, which keeps the divisor away from zero.

4 Vision Front End: From Pixels to a Time-to-Contact

4.1 Detection and tracking

`track_video` runs `ultralytics.YOLO(yolov8s.pt).track(..., conf=0.3, tracker=bytetrack.yaml, persist=True)`. The detection confidence threshold is 0.3 (the legacy HUD used 0.6) because a looming obstacle has low confidence at long range. The class filter is empty by default, since the COCO label assigned to a plain box is arbitrary. Frame time is $t = i/\mathrm{fps}$.

4.2 Target-track selection

The obstacle is taken to be the track with the largest accumulated bounding-box area,

$$\tau^\star = \arg\max_\tau \sum_k A_{\tau,k}, \qquad A = (x_2 - x_1)(y_2 - y_1), \tag{6}$$

on the grounds that a looming object dominates the view. All other tracks with ≥ 10 frames are reported (`other_tracks`) but not used.

4.3 Size signal

For each frame in which track $\tau^\star$ is seen, the box width and height w, h (pixels) give the scalar size

$$s = \sqrt{wh} \qquad (\text{with } wh \geq 10^{-9}). \tag{7}$$

4.4 Time-to-contact from monocular expansion

For a pinhole camera of focal length f viewing a fronto-parallel object of true linear size S at range Z, the apparent size is $s = fS/Z$, i.e. $1/s = Z/(fS)$. If the closing speed $v_c = -\dot{Z}$ is approximately constant,

$$\frac{1}{s(t)} = \frac{Z_0 - v_c t}{fS} \implies \frac{\mathrm{d}}{\mathrm{d}t}\Big(\frac{1}{s}\Big) = -\frac{v_c}{fS}, \qquad \boxed{\mathrm{TTC} = \frac{Z}{v_c} = -\frac{1/s}{\mathrm{d}(1/s)/\mathrm{d}t}} \tag{8}$$

which is independent of f and S (pixels suffice). Differencing two box heights is noisy, so the code fits a straight line by ordinary least squares (`np.polyfit`, degree 1) to $\{(t_k, 1/s_k)\}$ over a sliding window of the last $T_w = 0.75\,\mathrm{s}$ (requiring ≥ 5 samples):

$$\frac{1}{s} \approx \beta\, t + \alpha, \qquad \widehat{\beta} = \frac{\sum (t_k - \bar{t})(\sigma_k - \bar{\sigma})}{\sum (t_k - \bar{t})^2}, \quad \sigma_k = 1/s_k.$$

Evaluating the fit at the latest time t_n, $\sigma_n = \widehat{\beta} t_n + \widehat{\alpha}$, gives

$$\mathrm{TTC} = -\frac{\sigma_n}{\widehat{\beta}}, \qquad \dot{s} = -\frac{\widehat{\beta}}{\sigma_n^2} \quad (\text{since } \dot{s} = \tfrac{\mathrm{d}}{\mathrm{d}t}\sigma^{-1} = -\dot{\sigma}/\sigma^2). \tag{9}$$

TTC is reported only if $\widehat{\beta} < 0$ (approaching), $\sigma_n > 0$ and $\mathrm{TTC} \leq 30\,\mathrm{s}$; otherwise it is "not approaching" (a near-zero slope is noise). When the detector loses the object, the last fitted TTC is propagated for at most $T_{\mathrm{gap}} = 2\,\mathrm{s}$:

$$\mathrm{TTC}(t) = \max\big(0,\ \mathrm{TTC}_{\mathrm{last}} - (t - t_{\mathrm{last}})\big), \qquad \text{source} = \text{"extrapolated"}. \tag{10}$$

5 Metric Reconstruction (Pinhole Model)

Assumptions (all configurable, none measured): the real obstacle size $W \times H$ (default $0.5 \times 0.5\,\mathrm{m}$, thickness $0.05\,\mathrm{m}$) is supplied; the camera looks level along the forward axis with no distortion; the horizontal FOV is nominal (82.6°); the obstacle is fronto-parallel and static.

5.1 Focal length in pixels

For image width W_{img} and horizontal FOV α,

$$f = \frac{W_{\mathrm{img}}/2}{\tan(\alpha/2)}. \tag{11}$$

5.2 Range, lateral and vertical offsets

Width and height each give an independent range estimate, combined by their geometric mean:

$$Z_w = \frac{fW}{w}, \quad Z_h = \frac{fH}{h}, \quad Z = \sqrt{Z_w Z_h} = f\sqrt{\frac{WH}{wh}} = \frac{f\sqrt{WH}}{s}. \tag{12}$$

With image centre $(c_{x0}, c_{y0}) = (W_{\text{img}}/2, H_{\text{img}}/2)$ and box centre $(\bar{x}, \bar{y})$,

$$X = \frac{(\bar{x} - c_{x0})Z}{f} \text{ (right)}, \qquad Y = \frac{(c_{y0} - \bar{y})Z}{f} \text{ (up)}, \tag{13}$$

and the metric box size is $(wZ/f,\ hZ/f)$.

5.3 Closing speed

From $Z = k/s$ one gets $\dot{Z} = -Z\dot{s}/s$, hence the closing speed from the fitted growth rate $\dot{s}$ is

$$v_c = -\dot{Z} = \frac{Z\,\dot{s}}{s}. \tag{14}$$

Consistency with (**??**): $Z/v_c = s/\dot{s}$, which equals $-\sigma_n/\widehat{\beta}$ since $\dot{s} = -\widehat{\beta}/\sigma_n^2$ and $s = 1/\sigma_n$. During detection gaps flagged "extrapolated", $Z(t) = \max(0, Z_0 - v_0(t - t_0))$ with the last speed and lateral/vertical offsets held.

5.4 Scenario construction

The first $n_{\text{ref}} = 12$ measured rows having a speed are taken and the *median* (middle element of the sorted list) of range Z_0, closing speed v_0, lateral and vertical offsets is used:

$$\text{drone}: \ \boldsymbol{p}_0 = (0,0,0),\ \boldsymbol{v}_0 = (\max(v_0, 0), 0, 0),\ \text{heading } +x, \tag{15}$$

$$\text{obstacle centre}: \ (Z_0,\ -X_0,\ Y_0) \quad (\text{camera right} \to \text{world } -y), \tag{16}$$

$$\text{panel}: \ \text{width } W,\ \text{height } H,\ \text{thickness } t_p \text{ along } x,\ \text{normal } (-1,0,0), \tag{17}$$

$$\text{initial TTC}: \ Z_0/v_0 \ (\text{if } v_0 > 10^{-6}). \tag{18}$$

Note. The TTC of §4.4 is carried through for display; the planner consumes Z_0 and v_0 (i.e. the same information) rather than TTC itself.

5.5 Time scaling to the video

If the user overrides the approach speed (e.g. for a faster demonstration), the recording stores a `time_scale` $= v_{\text{video}}/v_{\text{sim}}$ so that video time is $t_{\text{video}} = t_{\text{ref}} + \text{time_scale} \cdot t_{\text{sim}}$ (`None` if not defined).

6 Vehicle Model (`params.py`, `motors.py`, `quadrotor.py`)

6.1 Parameters

The values below are *assumed, Tello-class orders of magnitude*, explicitly not a datasheet.

Derived constants:

$$k_T = \frac{T_{\max}}{\Omega_{\max}^2} = 1.778 \times 10^{-7}\ \text{N/(rad/s)}^2, \qquad k_M = \rho k_T = 2.844 \times 10^{-9}\ \text{N m/(rad/s)}^2, \tag{19}$$

$$u_{\text{hover}} = \sqrt{\frac{mg/4}{T_{\max}}} = 0.700, \qquad \Omega_{\text{hover}} = u_{\text{hover}}\Omega_{\max} \approx 1050\ \text{rad/s}. \tag{20}$$

The weight is $mg = 0.7845\,\text{N}$, so the thrust-to-weight ratio is $4T_{\max}/(mg) \approx 2.04$.

Quantity	Symbol	Value
Mass	m	$0.080\,\mathrm{kg}$
Arm offset (X layout, along x and y)	d	$0.045\,\mathrm{m}$
Inertia (diagonal)	$\mathrm{diag}(J_{xx}, J_{yy}, J_{zz})$	$(1,1,2)\times 10^{-4}\,\mathrm{kg\,m^2}$
Max rotor speed	$\Omega_{\max}$	$1500\,\mathrm{rad/s}$
Max thrust per motor	$T_{\max}$	$0.40\,\mathrm{N}$
Torque/thrust ratio	$\rho = k_M/k_T$	$0.016\,\mathrm{m}$
Rotor spin-axis inertia	J_r	$2\times 10^{-7}\,\mathrm{kg\,m^2}$
Motor time constant	τ_m	$0.03\,\mathrm{s}$
Motor efficiency, idle power	$\eta,\ P_0$	$0.7,\ 0.3\,\mathrm{W}$
Quadratic drag coefficient	c_d	$0.006\,\mathrm{kg/m}$
Angular damping	c_ω	$2\times 10^{-6}\,\mathrm{N\,m\,s}$
Tilt / acceleration limits (controller and planner)	$\theta_{\max}, a_{\max}$	$35^\circ,\ 6\,\mathrm{m/s^2}$

Table 2: `DroneParams` defaults.

6.2 Motor layout

X configuration; positions (x_i, y_i) and spin directions s_i (+1 CCW seen from above, −1 CW):

i	Position	x_i	y_i	s_i
1	front–right	$+d$	$-d$	$+1$
2	rear–right	$-d$	$-d$	-1
3	rear–left	$-d$	$+d$	$+1$
4	front–left	$+d$	$+d$	-1

6.3 Rotor and command model

Four quantities are kept separate: command u, angular speed Ω, thrust T and electrical power P.

$$u_i \in [0,1] \text{ (clipped)}, \qquad \Omega_i^{\mathrm{cmd}} = u_i\,\Omega_{\max} \quad \text{(linear map)}, \tag{21}$$

$$\dot{\Omega}_i = \frac{\Omega_i^{\mathrm{cmd}} - \Omega_i}{\tau_m} \quad \text{(first-order lag)}, \tag{22}$$

$$T_i = k_T \Omega_i^2, \qquad Q_i = k_M \Omega_i^2, \tag{23}$$

$$P_i = \frac{Q_i\,\Omega_i}{\eta} + P_0. \tag{24}$$

Because (**??**) is linear in u and (**??**) is quadratic in Ω, steady-state thrust is $T_i = T_{\max} u_i^2$; the inverse used by the controller is $u_i = \sqrt{\max(T_i, 0)/T_{\max}}$ (clipped to $[0,1]$). At hover, $Q = 3.14\times 10^{-3}\,\mathrm{N\,m}$ and $P \approx 5.0\,\mathrm{W}$ per motor ($\approx 20\,\mathrm{W}$ total). The rotor bandwidth is $1/\tau_m \approx 33\,\mathrm{rad/s}$.

6.4 Rigid-body equations of motion

The state is $\boldsymbol{x} = [\boldsymbol{p}, \boldsymbol{v}, \boldsymbol{q}, \boldsymbol{\omega}, \boldsymbol{\Omega}] \in \mathbb{R}^{17}$ $(3+3+4+3+4)$. Body torques from the rotors:

$$\tau_x = \sum_i y_i T_i, \qquad \tau_y = -\sum_i x_i T_i, \tag{25}$$

$$\tau_z = -\sum_i s_i (Q_i + J_r \dot{\Omega}_i). \tag{26}$$

The reaction torque on the body is opposite to the rotor spin; the $J_r\dot{\Omega}$ term is the reaction from accelerating the rotor. Translational dynamics, with aerodynamic drag on the *air-relative* velocity $\boldsymbol{v}_{\mathrm{rel}} = \boldsymbol{v} - \boldsymbol{w}$:

$$m\dot{\boldsymbol{v}} = \boldsymbol{R}(\boldsymbol{q}) \begin{bmatrix} 0 \\ 0 \\ \sum_i T_i \end{bmatrix} - mg\,\boldsymbol{e}_z - c_d \left\|\boldsymbol{v}_{\mathrm{rel}}\right\| \boldsymbol{v}_{\mathrm{rel}}, \qquad \dot{\boldsymbol{p}} = \boldsymbol{v}. \tag{27}$$

Rotational dynamics (Euler's equation with linear damping) and kinematics:

$$\boldsymbol{J}\dot{\boldsymbol{\omega}} = \boldsymbol{\tau} - \boldsymbol{\omega} \times \boldsymbol{J}\boldsymbol{\omega} - c_\omega \boldsymbol{\omega}, \qquad \dot{\boldsymbol{q}} = \tfrac{1}{2}\,\boldsymbol{q} \otimes [0, \boldsymbol{\omega}]. \tag{28}$$

The drag law is quadratic; with $c_d = \frac{1}{2}\rho_a C_d A$ the model corresponds to a lumped area–drag product. The drone's terminal free-fall speed is $\sqrt{mg/c_d} \approx 11.4\,\mathrm{m/s}$.

6.5 Numerical integration

`Quadrotor.step(u)` performs one fixed step ($h = \Delta t = 0.002\,\mathrm{s}$) of classical fourth-order Runge–Kutta with the motor command held constant (zero-order hold):

$$\boldsymbol{k}_1 = f(t, \boldsymbol{x}), \qquad \boldsymbol{k}_2 = f(t + \tfrac{h}{2}, \boldsymbol{x} + \tfrac{h}{2}\boldsymbol{k}_1), \tag{29}$$

$$\boldsymbol{k}_3 = f(t + \tfrac{h}{2}, \boldsymbol{x} + \tfrac{h}{2}\boldsymbol{k}_2), \qquad \boldsymbol{k}_4 = f(t + h, \boldsymbol{x} + h\boldsymbol{k}_3), \tag{30}$$

$$\boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \tfrac{h}{6}(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4), \tag{31}$$

followed by renormalising $\boldsymbol{q} \leftarrow \boldsymbol{q}/\left\|\boldsymbol{q}\right\|$. Time is $t = n\,\Delta t$ (an exact multiple of the step, independent of the wall clock). The wind function is evaluated at the stage times, so it must be a deterministic function of t. Logged outputs (command, Ω, T, P, body torque, acceleration, wind) are taken at the beginning of the step. The default initial rotor speeds are the hover speed (the drone starts airborne and in equilibrium).

6.6 Mixer

The steady-state map from per-motor thrusts to wrench $[F, \tau_x, \tau_y, \tau_z]^\top = \boldsymbol{A}[T_1, \ldots, T_4]^\top$ is

$$\boldsymbol{A} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ y_1 & y_2 & y_3 & y_4 \\ -x_1 & -x_2 & -x_3 & -x_4 \\ -s_1\rho & -s_2\rho & -s_3\rho & -s_4\rho \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -d & -d & d & d \\ -d & d & d & -d \\ -\rho & \rho & -\rho & \rho \end{bmatrix}. \tag{32}$$

(`thrusts_from_wrench` solves it numerically; it ignores the $J_r\dot{\Omega}$ term.) The rows are mutually orthogonal, with $\boldsymbol{A}\boldsymbol{A}^\top = \mathrm{diag}(4,\, 4d^2,\, 4d^2,\, 4\rho^2)$, so the inverse is analytic:

$$T_i = \frac{F}{4} + \frac{y_i\,\tau_x}{4d^2} - \frac{x_i\,\tau_y}{4d^2} - \frac{s_i\,\tau_z}{4\rho} = \frac{F}{4} \pm \frac{\tau_x}{4d} \pm \frac{\tau_y}{4d} \mp \frac{\tau_z}{4\rho}. \tag{33}$$

6.7 Actuation authority (derived)

Maximum roll/pitch torque (two motors at $T_{\max}$, two at 0) is $2dT_{\max} = 0.036\,\mathrm{N\,m}$, giving $\dot{p}_{\max} \approx 360\,\mathrm{rad/s^2}$; maximum yaw torque is $2\rho T_{\max} = 0.0128\,\mathrm{N\,m}$, giving $\dot{r}_{\max} = 64\,\mathrm{rad/s^2}$. Yaw authority is therefore $\sim$5.6$\times$ weaker than roll/pitch, which motivates the yaw-first sacrifice rule of §**??**.

7 Wind Model (`wind.py`)

7.1 Steady component

With azimuth ψ (from $+x$ toward $+y$) and elevation ε (from the horizontal plane toward $+z$), and describing where the wind blows *toward*,

$$\boldsymbol{w}_0 = V \begin{bmatrix} \cos\varepsilon\cos\psi \\ \cos\varepsilon\sin\psi \\ \sin\varepsilon \end{bmatrix}. \tag{34}$$

Thus $\psi = 0°$ is a tailwind, 180° a headwind, +90° blows toward the left, and $\varepsilon = 90°$ is a pure updraft. The inverse is $V = \|\boldsymbol{w}\|$, $\psi = \operatorname{atan2}(w_y, w_x)$, $\varepsilon = \arcsin(w_z/V)$. In the approach frame, headwind $= -w_x$, crosswind (left) $= w_y$, vertical $= w_z$.

7.2 Turbulence: independent Ornstein–Uhlenbeck processes

Each axis carries an OU (first-order Gauss–Markov) gust:

$$\mathrm{d}X = -\frac{X}{\tau}\mathrm{d}t + \sigma\sqrt{\frac{2}{\tau}}\,\mathrm{d}W_t. \tag{35}$$

Its exact discretisation over a step Δ is

$$X_{k+1} = a\,X_k + \sigma\sqrt{1-a^2}\,\xi_k, \qquad a = e^{-\Delta/\tau}, \quad \xi_k \sim \mathcal{N}(0,1), \tag{36}$$

started from the stationary law $X_0 \sim \mathcal{N}(0, \sigma^2)$. Therefore $\mathbb{E}[X] = 0$, $\mathrm{Var}[X] = \sigma^2$ and the autocorrelation is $R(\ell) = \sigma^2 e^{-|\ell|/\tau}$ *exactly*, independent of the sampling step Δ (here $\Delta = 0.01\,\mathrm{s}$). The corresponding one-sided power spectral density is the Lorentzian $S(f) = \dfrac{4\sigma^2\tau}{1+(2\pi f\tau)^2}$; this is a simple Gauss–Markov gust model, *not* the Dryden/von Kármán spectrum. Defaults: $\tau = 2\,\mathrm{s}$, $\sigma = 0$ (steady wind).

Implementation details that matter for reproducibility: the series is generated once on a fixed grid with a seeded `numpy.random.default`$_{r}ng(draworder : stationarystart, thenthenoiseblock), soalongertrackexter$ λX_{i+1}, and clamped outside $[0, t_{\max}]$. The total wind is $\boldsymbol{w}(t) = \boldsymbol{w}_0 + \boldsymbol{X}(t)$, which depends on time only (the argument position is ignored: there is no spatial shear).

8 Collision Geometry (`collision.py`)

The obstacle is an axis-aligned box $\mathcal{B}$ with centre $\boldsymbol{c}$ and half extents $\boldsymbol{h} = \frac{1}{2}[t_p, W, H]$ (thickness along x); the drone is a sphere of radius $r = 0.10\,\mathrm{m}$. Signed distance to the box surface:

$$\boldsymbol{q} = |\boldsymbol{p} - \boldsymbol{c}| - \boldsymbol{h}, \qquad \mathrm{sd}(\boldsymbol{p}) = \|\max(\boldsymbol{q}, \boldsymbol{0})\| + \min\big(\max_i q_i, 0\big), \tag{37}$$

positive outside, negative inside. Clearance is

$$\mathcal{C}(\boldsymbol{p}) = \min\Big(\mathrm{sd}(\boldsymbol{p}) - r, \quad p_z - z_{\texttt{floor}} - r\Big). \tag{38}$$

predict_collision marks a *collision* if $\mathcal{C} < 0$ anywhere on the sampled path and a *margin violation* if $\mathcal{C} < d_{\texttt{safe}}$ ($0.25\,\mathrm{m}$), and reports first-event times and $\min\mathcal{C}$. In the runner the obstacle's height z is offset by the start altitude (default $1.5\,\mathrm{m}$) so that it is level with the drone, and the floor is $z = 0$.

9 Avoidance Planner (`planner.py`)

The planner assumes the cruise heading is $+x$ at constant speed v. It works in the inertial frame, where the kinematics of the path are unaffected by wind; wind only changes the force the rotors must supply.

9.1 Minimum-jerk primitive

The quintic with zero velocity and acceleration at both ends:

$$\sigma(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \quad \sigma'(\tau) = 30\tau^2(1-\tau)^2, \quad \sigma''(\tau) = 60\tau - 180\tau^2 + 120\tau^3, \qquad \tau \in [0,1]. \tag{39}$$

Peaks: $\max \sigma' = \sigma'(\tfrac{1}{2}) = \tfrac{15}{8}$ and $\max|\sigma''| = \tfrac{10}{\sqrt{3}} \approx 5.77$.

9.2 Nominal path and key times

With start $\boldsymbol{p}_s$ and obstacle front/back planes $x_{\mathtt{f}}, x_{\mathtt{b}}$ inflated by body radius and safety margin,

$$x_{\mathtt{f}} = c_x - h_x - r - d_{\mathtt{safe}}, \qquad x_{\mathtt{b}} = c_x + h_x + r + d_{\mathtt{safe}}, \tag{40}$$

$$t_{\mathtt{f}} = \max\Big(\frac{x_{\mathtt{f}} - p_{s,x}}{v}, 0\Big), \qquad t_{\mathtt{b}} = \frac{x_{\mathtt{b}} - p_{s,x}}{v}. \tag{41}$$

The straight path is $\boldsymbol{p}_{\mathtt{nom}}(t) = \boldsymbol{p}_s + (vt, 0, 0)$, checked against the obstacle by `predict_collision`. If it keeps the safety margin, no avoidance is needed.

9.3 Detour family

A detour offsets the nominal path by D along a unit vector $\boldsymbol{u}(\phi) = (0, \cos\phi, \sin\phi)$ ($\phi = 0$: $+y$ left, $90°$: up), rises over T_b, holds, then returns:

$$s(t) = \sigma\Big(\frac{t - t_1}{T_b}\Big) - \sigma\Big(\frac{t - t_2}{T_b}\Big), \tag{42}$$

$$\boldsymbol{p}(t) = \boldsymbol{p}_{\mathtt{nom}}(t) + D\, s(t)\, \boldsymbol{u}, \quad \boldsymbol{v}(t) = \boldsymbol{v}_{\mathtt{nom}} + \frac{D}{T_b}\big[\sigma_r' - \sigma_f'\big]\boldsymbol{u}, \quad \boldsymbol{a}(t) = \frac{D}{T_b^2}\big[\sigma_r'' - \sigma_f''\big]\boldsymbol{u}, \tag{43}$$

with

$$t_1 = \max(0,\ e\, t_{\mathtt{f}} - T_b), \qquad t_2 = t_{\mathtt{b}} + 0.3\ \mathtt{s}. \tag{44}$$

The peak lateral acceleration of a detour is $a_{\mathtt{lat}} \approx D \cdot \frac{10}{\sqrt{3}}/T_b^2$ (e.g. $D = 0.7$, $T_b = 3\,\mathtt{s}$: $0.45\,\mathtt{m/s^2}$).

Candidate grid. 8 directions $\phi \in \{0, 45, \ldots, 315\}°$ × 19 offsets $D \in \{0.2, 0.3, \ldots, 2.0\}\,\mathtt{m}$ × 3 blend times $T_b \in \{1, 2, 3\}\,\mathtt{s}$ × 2 rise-end fractions $e \in \{0.6, 0.9\}$ = **912** candidates, sampled every $\Delta t = 0.02\,\mathtt{s}$.

9.4 Braking fallback

If no detour is feasible, a minimum-jerk speed profile removes the full speed over T:

$$\dot{x}(t) = v\big(1 - \sigma(t/T)\big), \quad x(t) = v\Big[t - T\big(\tfrac{5}{2}\tau^4 - 3\tau^5 + \tau^6\big)\Big], \ \tau = \frac{t}{T}, \quad a_x = -\frac{v}{T}\sigma'(\tau), \tag{45}$$

using $\int_0^\tau \sigma = \tfrac{5}{2}\tau^4 - 3\tau^5 + \tau^6$. The stopping distance is $vT/2$ (since $\int_0^1 \sigma = \tfrac{1}{2}$), so $T = 2\,d_{\mathtt{avail}}\kappa/v$ with $\kappa \in \{1, 0.9, 0.8\}$ and $d_{\mathtt{avail}} = x_{\mathtt{f}} - p_{s,x}$. Peak deceleration is $\tfrac{15}{8}v/T$. If this also fails, the result is "unavoidable within limits" and no avoidance flight is run.

9.5 Wind-aware feasibility

Air-relative drag acceleration is $\boldsymbol{a}_D = -(c_d/m)\,\|\boldsymbol{v}-\boldsymbol{w}\|\,(\boldsymbol{v}-\boldsymbol{w})$. To realise a planned inertial acceleration $\boldsymbol{a}_{\text{plan}}$, the rotors must supply

$$\boldsymbol{a}_{\text{req}} = \boldsymbol{a}_{\text{plan}} + \frac{c_d}{m}\,\|\boldsymbol{v}-\boldsymbol{w}\|\,(\boldsymbol{v}-\boldsymbol{w}), \qquad \boldsymbol{f} = \boldsymbol{a}_{\text{req}} + g\boldsymbol{e}_z \quad \text{(specific force)}, \tag{46}$$

and the limits are checked on this quantity (steady mean wind only):

$$\text{accel:} \quad \max\left\|\boldsymbol{a}_{\text{req}}\right\| \le \lambda a_{\max}, \tag{47}$$

$$\text{thrust:} \quad m\max\|\boldsymbol{f}\| \le \lambda \cdot 4T_{\max}, \tag{48}$$

$$\text{tilt:} \quad \max \arccos\!\left(f_z/\|\boldsymbol{f}\|\right) \le \lambda\,\theta_{\max}, \tag{49}$$

with $\lambda = 0.8$ (limit_fraction): effectively $4.8\,\text{m/s}^2$, $1.28\,\text{N}$ and $28°$. Clearance is checked on the plan with a required value $d_{\text{safe}}+0.05$ m$+\delta_{\text{extra}}$ (tracking buffer plus closed-loop re-plan buffer), against both obstacle and floor. Rejections are tallied by reason (a candidate may count under several).

Station keeping. At $\boldsymbol{v} = \boldsymbol{0}$ the air-relative velocity is $-\boldsymbol{w}$, so drag pushes the drone downwind and the rotors must supply

$$\boldsymbol{a}_{\text{req}} = -\frac{c_d}{m}\,\|\boldsymbol{w}\|\,\boldsymbol{w}, \qquad \text{tilt} = \arctan\frac{\|\boldsymbol{f}_{xy}\|}{f_z}, \tag{50}$$

(leaning *into* the wind; an updraft unloads the rotors). The maximum horizontal wind that can be held at the tilt limit follows from $c_d w^2 = mg\tan\theta_{\max}$: $w_{\max} = \sqrt{mg\tan 35°/c_d} \approx 9.6\,\text{m/s}$.

9.6 Selection cost

Among feasible candidates the planner minimises

$$J = D + 0.1\,\max\left\|\boldsymbol{a}_{\text{req}}\right\| + \begin{cases} 0.15 & \phi \in \{90°, 270°\}\ \text{(vertical)} \\ 0 & \text{otherwise,} \end{cases} \tag{51}$$

i.e. the smallest sufficient offset, with mild acceleration penalty, preferring sideways dodges (they keep altitude and use differential thrust). Order of preference if nothing is feasible: detour → brake → unavoidable.

10 Flight Controller (controller.py)

Cascade: *position/velocity (PD+I+feed-forward)* → desired force → *geometric attitude* → desired rates → *rate P loop with gyroscopic cancellation* → wrench → *desaturating mixer* → per-motor commands. The controller reads the true (noise-free) simulated state; it never reads the wind.

10.1 Disturbance observer (Luenberger on velocity)

Let $\boldsymbol{a}_T = \boldsymbol{R}\boldsymbol{e}_z\,T_{\text{meas}}/m - g\boldsymbol{e}_z$ be the thrust-plus-gravity acceleration, where $T_{\text{meas}} = \sum k_T\Omega_i^2$ is the rotor-speed (ESC-style) total thrust. The true velocity obeys $\dot{\boldsymbol{v}} = \boldsymbol{a}_T + \boldsymbol{d}$ with $\boldsymbol{d}$ the (drag) disturbance acceleration. The observer is

$$\dot{\widehat{\boldsymbol{v}}} = \boldsymbol{a}_T + \widehat{\boldsymbol{d}} + 2\omega_0(\boldsymbol{v} - \widehat{\boldsymbol{v}}), \tag{52}$$

$$\dot{\widehat{\boldsymbol{d}}} = \omega_0^2(\boldsymbol{v} - \widehat{\boldsymbol{v}}), \tag{53}$$

integrated by explicit Euler at the control step. With $\boldsymbol{e} = \boldsymbol{v} - \widehat{\boldsymbol{v}}$ and $\tilde{\boldsymbol{d}} = \boldsymbol{d} - \widehat{\boldsymbol{d}}$:

$$\dot{\boldsymbol{e}} = \tilde{\boldsymbol{d}} - 2\omega_0\boldsymbol{e}, \quad \dot{\tilde{\boldsymbol{d}}} = \dot{\boldsymbol{d}} - \omega_0^2\boldsymbol{e} \implies \ddot{\boldsymbol{e}} + 2\omega_0\dot{\boldsymbol{e}} + \omega_0^2\boldsymbol{e} = \dot{\boldsymbol{d}}. \tag{54}$$

This is critically damped with bandwidth $\omega_0 = 5\,\text{rad/s}$ (settling $\approx 4/\omega_0 \approx 0.8\,\text{s}$) and, for a constant $\boldsymbol{d}$, zero steady-state error (final-value theorem: a constant $\boldsymbol{d}$ gives $\dot{\boldsymbol{d}} = 0$ so $\boldsymbol{e} \to 0$ and $\widehat{\boldsymbol{d}} \to \boldsymbol{d}$). The estimate is norm-clipped to $a_{\max}$.

10.2 Position/velocity loop

With errors $\boldsymbol{e}_p = \boldsymbol{p}_d - \boldsymbol{p}$, $\boldsymbol{e}_v = \boldsymbol{v}_d - \boldsymbol{v}$ and the planner's acceleration feed-forward $\boldsymbol{a}_d$,

$$\boldsymbol{a}_{\mathrm{cmd}} = \boldsymbol{a}_d + \boldsymbol{K}_p \boldsymbol{e}_p + \boldsymbol{K}_d \boldsymbol{e}_v + \boldsymbol{\iota} - \widehat{\boldsymbol{d}}, \tag{55}$$

with $\boldsymbol{K}_p = \mathrm{diag}(4,4,8)\,\mathrm{s}^{-2}$, $\boldsymbol{K}_d = \mathrm{diag}(3.5,3.5,5)\,\mathrm{s}^{-1}$ and integral gains $\boldsymbol{K}_i = \mathrm{diag}(0.5,0.5,1)$. The integral state is

$$\boldsymbol{\iota} \leftarrow \mathrm{clip}\big(\boldsymbol{\iota} + \boldsymbol{K}_i \boldsymbol{e}_p \,\Delta t,\ \pm 1.5\big) \tag{56}$$

updated only if $\|\boldsymbol{e}_p\| < 0.3\,\mathrm{m}$ ***and*** the mixer was not saturated on the previous step (anti-windup). When the observer is active, $\boldsymbol{K}_i$ is scaled by 0.05 (the observer removes the constant disturbance; the integrator is only a slow residual trim).

Each axis is a second-order loop $\ddot{e} + K_d\dot{e} + K_p e = 0$: for x, y, $\omega_n = \sqrt{4} = 2\,\mathrm{rad/s}$, $\zeta = 3.5/(2\cdot 2) = 0.875$; for z, $\omega_n = \sqrt{8} \approx 2.83$, $\zeta = 5/(2\sqrt{8}) \approx 0.88$.

10.3 Acceleration and tilt limiting

1. If $\|\boldsymbol{a}_{\mathrm{cmd}}\| > a_{\max}$ rescale to $a_{\max}$.
2. $\boldsymbol{F} = m(\boldsymbol{a}_{\mathrm{lim}} + g\boldsymbol{e}_z)$, with $F_z \leftarrow \max(F_z, 0.2\,mg)$ (keep upright).
3. If $\|\boldsymbol{F}_{xy}\| > F_z \tan\theta_{\max}$, scale $\boldsymbol{F}_{xy}$ down to that bound.

Either clipping sets the limited flag.

10.4 Desired attitude and collective thrust

Body z axis is along the force; heading comes from the yaw set-point ψ_d (always 0 in the runner):

$$\boldsymbol{z}_b = \frac{\boldsymbol{F}}{\|\boldsymbol{F}\|}, \quad \boldsymbol{x}_c = [\cos\psi_d, \sin\psi_d, 0]^\top, \quad \boldsymbol{y}_b = \frac{\boldsymbol{z}_b \times \boldsymbol{x}_c}{\|\boldsymbol{z}_b \times \boldsymbol{x}_c\|}, \quad \boldsymbol{x}_b = \boldsymbol{y}_b \times \boldsymbol{z}_b, \tag{57}$$

$$\boldsymbol{R}_d = [\boldsymbol{x}_b\ \ \boldsymbol{y}_b\ \ \boldsymbol{z}_b], \qquad T_c = \max\big(\boldsymbol{F}\cdot\boldsymbol{R}\boldsymbol{e}_z,\ 0\big). \tag{58}$$

The collective thrust is the projection of the desired force on the *current* body axis.

10.5 Geometric attitude error (Lee–Leok–McClamroch)

$$\boldsymbol{e}_R = \tfrac{1}{2}\big(\boldsymbol{R}_d^\top \boldsymbol{R} - \boldsymbol{R}^\top \boldsymbol{R}_d\big)^\vee, \qquad \boldsymbol{\omega}_d = \mathrm{clip}\big(-\boldsymbol{K}_{\mathrm{att}}\boldsymbol{e}_R,\ \pm 8\ \mathrm{rad/s}\big), \tag{59}$$

where $(\cdot)^\vee$ extracts $[M_{32}, M_{13}, M_{21}]$ from the skew matrix, and $\boldsymbol{K}_{\mathrm{att}} = \mathrm{diag}(7,7,3)\,\mathrm{s}^{-1}$. This avoids Euler-angle singularities and is almost-globally convergent.

10.6 Rate loop and gyroscopic cancellation

$$\boldsymbol{\tau} = \boldsymbol{J}\big[\boldsymbol{K}_\omega(\boldsymbol{\omega}_d - \boldsymbol{\omega})\big] + \boldsymbol{\omega} \times \boldsymbol{J}\boldsymbol{\omega}, \qquad \boldsymbol{K}_\omega = \mathrm{diag}(18,18,10)\ \ \mathrm{s}^{-1}. \tag{60}$$

Substituted into (??) (ignoring damping) this yields $\dot{\boldsymbol{\omega}} = \boldsymbol{K}_\omega(\boldsymbol{\omega}_d - \boldsymbol{\omega})$: a first-order rate loop of bandwidth $18\,\mathrm{s}^{-1}$ inside a $7\,\mathrm{s}^{-1}$ attitude loop, separating time scales as required for cascade stability. The motor lag ($\approx 33\,\mathrm{rad/s}$) is the next, faster, element.

10.7 Desaturating mixer

Given $(T_c, \boldsymbol{\tau})$ find $T_i \in [0, T_{\max}]$. By (??), $T_i = a_i(\boldsymbol{\tau}) + c$ where $\boldsymbol{a} = \boldsymbol{A}^{-1}[0, \boldsymbol{\tau}]^\top$ is zero-sum and $c = F/4$ is the common-mode thrust. Define the span $\Delta(\boldsymbol{\tau}) = \max_i a_i - \min_i a_i$. Priority: **roll/pitch > thrust > yaw**.

1. If $\Delta(\boldsymbol{\tau}) > T_{\max}$, bisect (30 iterations) the yaw scale $\mu \in [0,1]$ so that $\Delta([\tau_x, \tau_y, \mu\tau_z]) \le T_{\max}$, and set $\tau_z \leftarrow \mu\tau_z$.

2. If still $\Delta > T_{\max}$, scale roll and pitch uniformly: $\tau_{x,y} \leftarrow \tau_{x,y}\, T_{\max}/\Delta$.
3. Clip the common mode into the feasible range: $c = \mathrm{clip}\big(T_c/4,\ -\min_i a_i,\ T_{\max} - \max_i a_i\big)$.

The saturated flag is set whenever step 1 or step 3 intervenes. The output commands are $u_i = \sqrt{T_i/T_{\max}}$ and the used thrust $4c$ feeds back (as the previous-step thrust fallback).

11 Closed-Loop Execution (`loop.py`, `runner.py`)

11.1 Multi-rate loop

Physics runs at 500 Hz ($\Delta t = 0.002$ s); the controller at 250 Hz, i.e. every per_ctrl $= \mathrm{round}\big(1/(f_c \Delta t)\big) = 2$ physics steps, with zero-order hold between updates. History is recorded every 2 steps (250 Hz).

11.2 Flight set-point from the plan

The set-point is $(\boldsymbol{p}_d, \boldsymbol{v}_d, \boldsymbol{a}_d)(t)$ from linear interpolation of the planned trajectory (constant-velocit extrapolation after the end), with yaw 0.

11.3 Pre-roll

When wind is active, the drone first flies, unrecorded, along the approach line for 6 s at *negative* time (gust series held at its $t = 0$ value), starting from $\boldsymbol{p}_s - \boldsymbol{v}_0 \cdot 6$, so that at $t = 0$ the drone and its observer are in equilibrium with the wind. Without wind there is no pre-roll.

11.4 Plan–fly–verify–re-plan

1. Plan with the mean wind; if avoidance is needed and enabled, also fly the straight path ("nominal flight") to show what would have happened.
2. Fly the chosen plan for $t_{\text{end}} + 1$ s and compute $\mathcal{C}_{\min}$ on the *flown* path.
3. If turbulence is on and $\mathcal{C}_{\min} < d_{\text{safe}}$ and re-plans ≤ 2 were used, set

$$\delta_{\text{extra}} \leftarrow \delta_{\text{extra}} + (d_{\text{safe}} - \mathcal{C}_{\min}) + 0.05, \tag{61}$$

re-plan, and fly again. If the planner cannot add that clearance, keep the last verified flight.

Summary metrics: minimum flown clearance and its time; collision flag ($\mathcal{C}_{\min} < 0$); maximum tilt $\arccos R_{33}$; peak speed; maximum tracking error $\max_t \|\boldsymbol{p} - \boldsymbol{p}_d\|$; fraction of time saturated; fraction limited.

12 Recording, Replay, API and Front End

12.1 Recording (`sim_recording/v1`)

Contents: physics settings, frames, drone parameters (k_T, k_M, hover command, motor layout), environment (box centre/half extents, margin, floor), scenario, planning diagnostics (candidates, feasible, rejections, plan parameters, nominal trajectory), wind block (with the station-keeping requirement), and two flight blocks (flight, nominal_flight) each with time, position, velocity, quaternion, Euler angles, body rates, motor command/Ω/thrust/power, set-point, clearance, flags, wind and observer estimate. Floats are rounded to 6 decimals and no wall-clock data are stored, so identical runs are byte-identical.

12.2 Replay

`Replay.at(t)` is a pure function of t (any scrubbing order gives the same answer). With sample index i and $j = i+1$, $\lambda = (t - t_i)/(t_j - t_i)$:

- positions, velocities, rates, Ω, thrust, clearance, wind: $x = (1-\lambda)x_i + \lambda x_j$;
- attitude: normalised lerp (NLERP), flipping $\boldsymbol{q}_j \to -\boldsymbol{q}_j$ if $\boldsymbol{q}_i \cdot \boldsymbol{q}_j < 0$ (shortest arc), $\boldsymbol{q} = \frac{(1-\lambda)\boldsymbol{q}_i + \lambda \boldsymbol{q}_j}{\|\cdot\|}$;
- motor commands: zero-order hold at sample i (matching how the controller applied them);
- airspeed $= \|\boldsymbol{v} - \boldsymbol{w}\|$; video time $= t_{\texttt{ref}} + \texttt{time_scale} \cdot t$.

A TypeScript port (`replay.ts`) is verified against a golden JSON fixture generated by `tools/make_re`

12.3 API

FastAPI endpoints: `GET /api/health`, `/api/runs`, `/api/videos`, `/api/runs/{run}/{track|metric|recordi` `POST /api/videos/{name}/process` (async job running the three pipeline steps, polled via `GET /api/jobs/{id}`) and `POST /api/runs/{run}/simulate` (re-simulation with validated parameter ranges: speed 0.05–$15\,\texttt{m/s}$, wind $\le 30\,\texttt{m/s}$, turbulence $\sigma \le 5\,\texttt{m/s}$, $\tau \in [0.2, 30]\,\texttt{s}$, ...). Run names are validated against a strict regex and resolved inside the data directory (path-traversal guard).

12.4 Front-end mathematics

Attitude is shown in aviation convention. Simulator pitch (about $+y$) is positive nose-*down*; the HUD therefore displays $\texttt{pitch}_{\uparrow} = -\theta$, $\texttt{roll}_{\texttt{right}} = \phi$, $\texttt{yaw} = \psi$. The artificial horizon shifts by $\min(28, \max(-28, 1.4\,\texttt{pitch}_{\uparrow}))$ px and rotates by $-\texttt{roll}_{\texttt{right}}$. The wind widgets use the same $(V, \psi, \varepsilon) \leftrightarrow \boldsymbol{w}$ maps as §7, kept in sync by unit tests. Motor graphs use min/max decimation per pixel column (preserving local extrema), padded y-ranges with a minimum span, and "nice" tick positions with step $\in \{1, 2, 5, 10\} \times 10^{\lfloor \log_{10}(\texttt{raw}) \rfloor}$.

13 Worked Numerical Examples from the Stored Runs

Both examples read from `data/runs/*/sim_recording.json`.

13.1 Run `testv2` (slow approach)

Reconstruction: $Z_0 = 2.715\,\texttt{m}$, $v_0 = 0.328\,\texttt{m/s}$, initial TTC $= 8.28\,\texttt{s}$. Planner times (obstacle half-thickness 0.025, $r = 0.10$, $d_{\texttt{safe}} = 0.25$):

$$x_{\texttt{f}} = 2.715 - 0.025 - 0.10 - 0.25 = 2.340\ \texttt{m}, \quad t_{\texttt{f}} = 7.14\ \texttt{s}, \quad t_{\texttt{b}} = \frac{2.715 + 0.375}{0.328} = 9.43\ \texttt{s}, \quad t_2 = 9.73\ \texttt{s},$$

and with $e = 0.9$, $T_b = 3$: $t_1 = 0.9 \cdot 7.14 - 3 = 3.43\,\texttt{s}$. Both match the recorded plan (`rise_start_s`= 3.43, `return_start_s`= 9.73). The chosen plan: left ($\phi = 0$), $D = 0.7\,\texttt{m}$, $T_b = 3\,\texttt{s}$, cost 0.745; 114 candidates evaluated, 60 feasible. Outcome: flown minimum clearance $0.349\,\texttt{m}$ (no collision) vs. $-0.125\,\texttt{m}$ for the straight flight (collision); maximum tilt $3.4°$; maximum tracking error $0.020\,\texttt{m}$.

13.2 Run `test2` (fast approach)

$Z_0 = 4.107\,\texttt{m}$, $v_0 = 4.063\,\texttt{m/s}$, initial TTC $\approx 1.01\,\texttt{s}$. Chosen: $\phi = 135°$ (up and toward $-y$), $D = 0.4\,\texttt{m}$, $T_b = 1\,\texttt{s}$; 912 candidates, 114 feasible, with rejection tallies accel 256, thrust 104, tilt 204, clearance 566, floor 86 (a candidate can fail several tests). Flown clearance

0.335 m (no collision) vs. −0.099 m straight; maximum tilt 19.5°; tracking error 0.146 m. Here the planner's acceleration, thrust and tilt checks genuinely bind: a 1 s blend at this speed demands $a \sim D \cdot 5.77/T_b^2 \approx 2.3\,\mathrm{m/s^2}$ lateral and a large drag contribution ($c_d v^2/m \approx 1.2\,\mathrm{m/s^2}$).

14 Derived Analytical Quantities

Quantity	Value
Weight	0.785 N
Max total thrust	1.60 N ($T/W \approx 2.04$)
Hover command / rotor speed	0.700 / 1050 rad/s
Hover electrical power	≈ 5.0 W per motor
Max roll/pitch angular acceleration	360 rad/s^2
Max yaw angular acceleration	64 rad/s^2
Terminal speed (free fall, still air)	11.4 m/s
Drag at 4 m/s	0.096 N (≈ 12% of weight)
Max holdable horizontal wind at 35°	≈ 9.6 m/s
Planner limits at $\lambda = 0.8$	4.8 m/s^2, 1.28 N, 28°
Position loop (ω_n, ζ) x,y / z	$(2, 0.875)$ / $(2.83, 0.88)$
Observer bandwidth / settling	5 rad/s / ≈ 0.8 s
Loops: rate / attitude / motor	18 / 7 / 33 s^{-1}

Table 3: Quantities computed from the default parameters.

15 Assumptions, Limitations and Code Observations

1. **README is out of date.** It describes the legacy HUD (heuristic distance, (??)); the vision → planner → controller → physics → web replay chain is not documented there.
2. **All physical constants are assumed.** Mass, inertia, thrust, drag, FOV and obstacle size are placeholders, flagged by size_is_assumed and fov_is_assumed. The metric output is therefore only as good as these.
3. **Perfect state, no sensors.** The controller reads the true state (no IMU/vision noise, delay or bias); only rotor-derived thrust is treated as a "measurement".
4. **Static, single, box obstacle.** The planner uses env.boxes[0] only, assumes cruise heading $+x$ and constant speed, and a spherical drone. TTC from video is displayed but the planner uses (Z_0, v_0) from the first 12 measured rows (a one-shot scenario, not online replanning from live vision).
5. **Wind in planning vs. flight.** Planning uses only the steady mean; gusts act in flight and are handled by closed-loop verification and re-planning ($\leq$ 2 attempts). The wind has no spatial dependence and does not affect rotors (no inflow/blade-flapping effects), and the observer lumps all unmodelled acceleration into $\hat{\boldsymbol{d}}$.
6. **Simplified rotor/aero physics.** No gyroscopic rotor torque (only the $J_r\dot{\Omega}$ reaction), no ground effect, no propeller-induced drag, no body-shape-dependent drag (single scalar c_d).
7. **Mixer ignores spin-up torque**, and the desaturation prioritises attitude over yaw and thrust (an intentional design choice); a saturated flag is reported.
8. **Euler integration of observer / integrator** at 250 Hz is adequate for the chosen bandwidths ($\omega_0 \Delta t = 0.02$).
9. **Minor text inconsistency.** The recording's assumptions list still ends with "Wind: none." even though a wind block is written; the wind section is authoritative.
10. **Candidate cost** treats up/down dodges as only 0.15 m more expensive than sideways, so a vertical dodge is chosen when sideways is infeasible or much larger.

A Pseudo-code of One Control Cycle

1. $T_{\texttt{meas}} \leftarrow k_T \sum \Omega_i^2$; $\boldsymbol{a}_T \leftarrow \boldsymbol{R}\boldsymbol{e}_z T_{\texttt{meas}}/m - g\boldsymbol{e}_z$.
2. `Observer:` $\boldsymbol{e} \leftarrow \boldsymbol{v} - \widehat{\boldsymbol{v}}$; $\widehat{\boldsymbol{v}} \mathrel{+}= \Delta t(\boldsymbol{a}_T + \widehat{\boldsymbol{d}} + 2\omega_0 \boldsymbol{e})$; $\widehat{\boldsymbol{d}} \mathrel{+}= \Delta t\, \omega_0^2 \boldsymbol{e}$; `clip` $\left\|\widehat{\boldsymbol{d}}\right\| \le a_{\max}$.
3. `Errors` $\boldsymbol{e}_p, \boldsymbol{e}_v$; `update` $\boldsymbol{\iota}$ `(anti-windup conditions).`
4. $\boldsymbol{a}_{\texttt{cmd}} \leftarrow \boldsymbol{a}_d + \boldsymbol{K}_p \boldsymbol{e}_p + \boldsymbol{K}_d \boldsymbol{e}_v + \boldsymbol{\iota} - \widehat{\boldsymbol{d}}$; `limit magnitude.`
5. $\boldsymbol{F} \leftarrow m(\boldsymbol{a} + g\boldsymbol{e}_z)$; `floor` F_z; `tilt-limit.`
6. `Build` $\boldsymbol{R}_d$; $T_c \leftarrow \boldsymbol{F} \cdot \boldsymbol{R}\boldsymbol{e}_z$.
7. $\boldsymbol{e}_R$; $\boldsymbol{\omega}_d \leftarrow \mathrm{clip}(-\boldsymbol{K}_{\texttt{att}} \boldsymbol{e}_R)$.
8. $\boldsymbol{\tau} \leftarrow \boldsymbol{J}\boldsymbol{K}_\omega(\boldsymbol{\omega}_d - \boldsymbol{\omega}) + \boldsymbol{\omega} \times \boldsymbol{J}\boldsymbol{\omega}$.
9. `Desaturate` $(T_c, \boldsymbol{\tau}) \to T_i$; $u_i \leftarrow \sqrt{T_i / T_{\max}}$ `(clipped).`
10. `Hold` $\boldsymbol{u}$ `for 2 physics steps of RK4.`

B Reproducing the Run

```
python -m dronesim.vision.export  testv2.mp4 --out data/runs/testv2
python -m dronesim.vision.reconstruction data/runs/testv2/obstacle_track.json
python -m dronesim.run_scenario data/runs/testv2/obstacle_metric.json \
        --wind-speed 5 --wind-azimuth 180 --turbulence 0.5 --wind-seed 1
python -m dronesim.api        # http://127.0.0.1:8000 (docs at /docs)
```